R.Poojitha
192311255

CO2-1

Bubble Sort-Mixed Increasing and Decreasing Blocks

Given Dataset: [8,16,24,32,20,12]

The Sequence Contains:

* Increasing Block: 8→16→24→32
* Decreasing Block: 32→20→12

The Disorder Occurs because 20 and 12 are smaller than the Elements before them. Bubble Sort resolves this by Repeatedly Comparing Adjacent Elements and Swapping them whenever the Left Element is Greater than the Right Element.

Pass-by-pass Sorting

we use Ascending Bubble Sort

Pass 1: Start-[8,16,24,32,20,12]

Comparison	Result
8,16	No swap
16,24	NoSwap
24,32	No swap
32,20	Swap
32,12	Swap

After pass 1:

[8, 16, 24, 20, 12, 32]

The Largest Element 32, has bubbled to the end.

Pass 2 [8, 16, 24, 20, 12, 32

Comparison	Result
8,16	No swap
16,24	No swap
24,20	No swap
20,12	Swap
12,32	Swap

After pass 2: [8, 16, 20, 12, 24, 32]

Pass 3 [8, 16, 20, 12, 24, 32]

Comparison	Result
8,16	No swap
16,20	No swap
20, 12	Swap

After pass 3: [8, 16, 12, 20, 24, 32]

Therefore,

Total Comparisons = $5+4+3+2+1=15$

Total Swaps = $2+2+1+1=6$

So, Comparisons $=15$, Swaps $=6$

The 6 swaps also correspond to the 6 inversions in the original sequence

Why Bubble sort works well here:

→ The Four Elements are already Increasing

$8<16<24<32$

The problem is the decreasing tail

$32>20>12$

Bubble sort Gradually moves the smaller Elements toward the Begining while pushing Larger Numbers to the End.

Time Complexity

Case	Comparisons	Time Complexity
Best Case	$O(n)$	$O(n)$
Average Case	$O(n^2)$	$O(n^2)$
Worst Case	$O(n^2)$	$O(n^3)$

Bubble sort is a Brute force Comparison-Based Technique. It repeatedly Examines Adjacent Elements and removes Local disorder through Swaps.

Pass 4 : [8, 16, 12, 20, 24, 32]

Comparison	Result
8,16	No swap
16,12	Swap

After Swap pass 4 : [8, 12, 16, 20, 24, 32]

Pass 5 : [8, 12, 16, 20, 24, 32]

All remaining Elements are already ordered

After pass 5:

[8, 12, 16, 20, 24, 32]

No swaps Occur, So an optimized Bubble Sort terminates here.

Total Comparisons and Swaps

For the Standard Bubble sort Implementation:

* Pass 1 : 5 Comparisons, 2 Swaps
* pass 2 : 4 Comparisons, 2 swaps
* Pass 3 : 3 Comparisons, 1 swap
* pass 4 : 2 Comparisons, 1 swap
* pass 5 : 1 Comparison, 0 swaps